Solid Developer – Movement Assessment

Company: Bruno Vehicle Hire

Scope: Full System (Vehicles + Customers + Bookings)

---

1. Purpose of the Assessment

The purpose of this assessment is to evaluate the candidate's ability to:

- Architect a small system end-to-end
- Model business rules properly
- Apply DDD concepts
- Implement cross-entity constraints
- Build scalable frontend structure
- Handle validation and edge cases
- Demonstrate production-level thinking

---

2. Domain Model

Entities

Vehicle

- Id (Guid)
- RegistrationNumber (unique)
- Make
- Model
- Year
- DailyRate
- IsDeleted
- CreatedDate

Soft delete required.

Customer

- Id (Guid)
- FirstName
- LastName
- Email (unique)
- PhoneNumber
- CreatedDate

Deletion allowed only if no bookings exist.

Booking

- Id (Guid)
- VehicleId (FK)
- CustomerId (FK)
- StartDate
- EndDate
- TotalPrice
- Status (Active / Completed / Cancelled)
- CreatedDate

---

3. Business Rules

Vehicles

- Soft delete only
- Cannot book a soft-deleted vehicle
- Cannot book overlapping date ranges

Customers

- Cannot delete if bookings exist

Bookings

- Cannot overlap bookings for same vehicle
- EndDate must be greater than StartDate
- Cannot delete past bookings
- Can delete only if booking is in the future

---

4. Technical Requirements (Backend)

Architecture

- Clean Architecture enforced strictly
- Domain layer:
  - Rich domain models
  - Business rules inside entities
  - Value Objects (e.g., DateRange)
- Application layer:
  - Commands / Queries
  - DTOs
  - Validation
- Infrastructure:
  - EF Core
  - Repository implementations
- API:
  - Thin controllers

Required Patterns

- Repository Pattern
- CQRS
- MediatR
- Domain Events (bonus)
- FluentValidation
- EF Core Code-first migrations
- Unit of Work (optional but recommended)

Database

- SQL Server / PostgreSQL
- Proper indexing
- Foreign keys enforced

- Unique constraints
- Soft delete filtering via query filters

API

- RESTful endpoints for all 3 entities
- API Key authentication
- Global exception handling middleware
- Logging
- Pagination + filtering
- Swagger with examples

---

5. Frontend Requirements – Solid Developer

At Solid level, the expectation shifts from “can build CRUD” to:

- Can design scalable frontend architecture
- Can structure feature modules
- Can manage state effectively
- Can handle cross-entity interactions
- Can build production-quality UI patterns

The candidate may choose:

- Angular
- React (Web)
- React Native
- Flutter

General Solid-Level Expectations (All Frameworks)

Required Functional Features

- Full Vehicles module
- Full Customers module
- Full Bookings module
- Filtering & search
- Date handling

- Business rule feedback (e.g., overlapping bookings)
- Centralized error handling
- Reusable modals
- Confirmation dialogs
- API key handling securely
- Proper environment configuration

Architecture Expectations (Solid Level)

Must demonstrate:

- Feature-based structure (NOT entity-by-entity pages dumped in root)
- Centralized API client
- Separation of:
    - Domain models
    - DTOs
    - View models
- Reusable shared components
- Clear state management approach
- Scalable folder layout

Framework-Specific Expectations (choose 1)

5.1 React (Web)

Required Stack

- React + TypeScript
- Vite
- React Router
- Axios
- React Query (required)
- React Hook Form
- Zod/Yup validation

Expected Architecture

```
src/
 ├── app/
 ├── features/
 |    ├── vehicles/
 |    ├── customers/
 |    └── bookings/
 ├── shared/
 |    ├── components/
 |    ├── hooks/
 |    ├── api/
 |    └── utils/
 └── main.tsx
```

Expected Competencies

- Query caching with React Query
- Mutations with invalidation
- Error boundaries
- Loading skeletons
- Avoid prop drilling
- Composable hooks

5.2 Angular

Required

- Feature modules per entity
- Shared module
- Angular services abstraction
- RxJS operators properly used
- Reactive Forms
- Route guards (bonus)
- Interceptor for API Key

Architecture

src/app/

├── core/

├── shared/

├── features/

│ ├── vehicles/

│ ├── customers/

│ └── bookings/

Expected Competencies

- Smart vs presentational components
- Observable streams
- Form group validation
- Error interceptor
- Proper module boundaries

5.3 React Native

Required

- TypeScript
- React Navigation
- Axios
- React Query (required)
- React Hook Form
- Central API client

Expected

- Modular screen grouping
- Proper navigation stacks
- Date picker integration
- State normalization
- Async loading UX

Architecture

src/

├── features/

├── navigation/

├── shared/

├── api/

└── App.tsx

5.4 Flutter

Required

- Feature-based structure
- State management (Riverpod or Bloc required at Solid level)
- Dio for API abstraction
- Proper error handling
- Form validation

Architecture

lib/

├── core/

├── features/

│ ├── vehicles/

│ ├── customers/

│ └── bookings/

├── shared/

└── main.dart

Expected Competencies

- Separation of presentation & logic
- Proper async/await usage
- Clean widget tree
- Reusable form components
- Strongly typed models

---

6. Non-Functional Requirements

- Unit tests (minimum):
    - 2 Commands
    - 2 Queries
    - 1 business rule test
- Meaningful commit history
- Proper error handling
- Clean README with:
    - Architecture explanation
    - How to run
    - Assumptions made
- Seed data

---

7. Evaluation Focus (Solid)

- System design thinking
- Domain modelling quality
- Correct enforcement of business rules
- Architectural consistency
- Frontend modularization
- API quality
- Edge case handling
- Code maintainability